Technische Universität Berlin

Distributed Systems
Chapter 2: Cooperation and Coordination

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

2.1 Fundamental System Models

2.2 Cooperation

2.3 Coordination

2.1 Fundamental System Models

System models help us to

- Understand whether a design will work
- Understand and prove important properties of the system (e.g. using math models)
- Clarify understanding by abstracting unnecessary details away (e.g. hardware)

Models provide an abstract and simplified description of a certain DS aspect

- Physical model: Capture hardware composition in terms of nodes and interconnecting networks

System Models - Introduction

System models help us to:

- Understand whether a design will work
- Understand and prove important properties of the system (e.g. using math models)
- Clarify understanding by abstracting unnecessary details away (e.g. hardware)

Models provide an abstract and simplified description of a certain DS aspect

- Physical model:
- Architectural model: Describes a system in terms of the tasks performed by its elements to interact with each other (e.g. request-reply in a client / server system)

System Models - Introduction

System models help us to:

- Understand whether a design will work
- Understand and prove important properties of the system (e.g. using math models)
- Clarify understanding by abstracting unnecessary details away (e.g. hardware)

Models provide an abstract and simplified description of a certain DS aspect

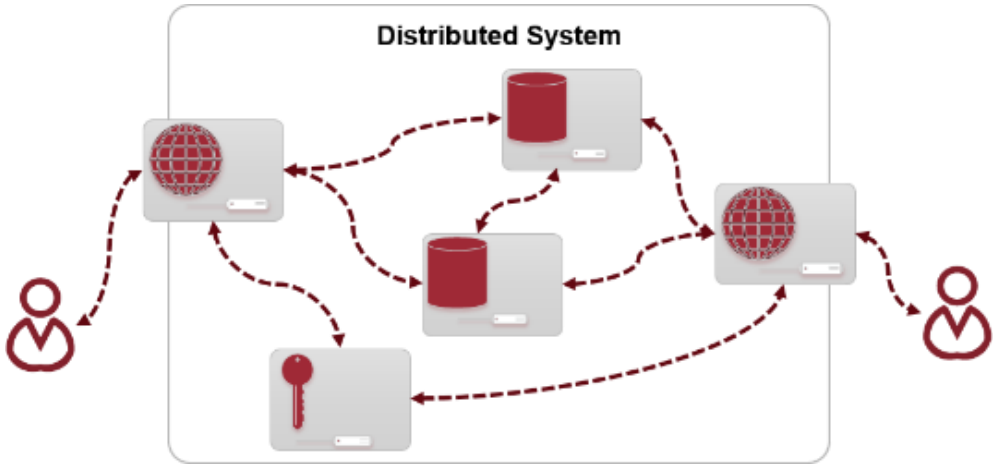
- Physical model:
- Architectural model:
- Fundamental model(s): Formal description of the properties shared by all architectural models

Physical Model

A representation of the underlying hardware elements: Abstracts from specific details of the used computer and networking technologies

Baseline Physical Model:

- Based on the initial DS definition
- An extensible set of computer nodes interconnected by a network for the required passing of messages
- More descriptive models can be derived



Physical Model - (Generations of) Physical Models

Early DS:

- 10-100 nodes interconnected by a local area network (LAN)
- Support a small range of (Intranet) services like shared printers or files

Internet-scale DS:

- Extensible set of nodes interconnected by a network of networks (Internet)
- Provide services for global organizations and across organizational boundaries (e.g. Google Search Engine)
- Significant level of heterogeneity in terms of networks, computer architecture, operating systems, …

Physical Model

Nodes in both generations of physical models are

- Static: remain in one physical location
- Discrete: not embedded in other physical entities (e.g. smart TV)
- Autonomous: independent of other nodes in terms of physical infrastructure and given role

Contemporary DS:

- Nodes move from location to location: service discovery, ad-hoc comm required
- Embedded into everyday objects
- Autonomous nodes with distinct roles are transformed to pools of nodes that together provide services

Architectural Model

The architectural model defines

- What are the communicating entities in DS?
- How do they communicate or what are the communication paradigms used?
- What roles and responsibilities do the entities have in the overall architecture?
- How are the entities mapped on to the physical infrastructure or what is their placement?

Architectural Model - Communicating entities

System-oriented perspective: entities = processes coupled with inter-process communication paradigms

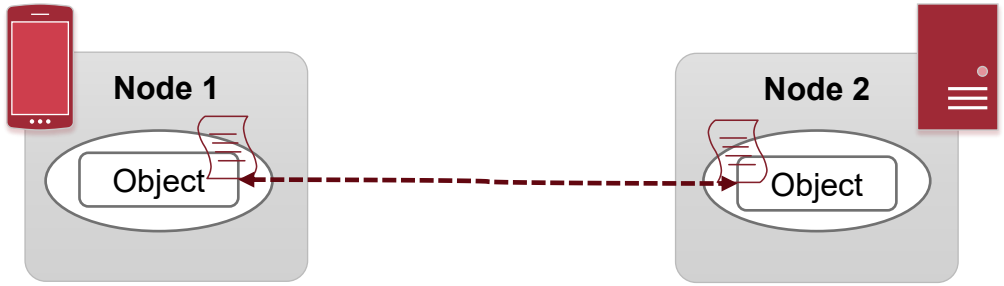
Special cases: OS does not support a process abstraction (e.g. constrained environments like sensor networks) → entity = node

Architectural Model - Communicating entities

Problem-oriented perspective: instead of focusing on system model's point of view, focus on the developer perspective

Object-oriented approaches in DS

- A computation consists of interacting objects, each encapsulating a part of the problem domain
- Objects are accessed and described through interfaces

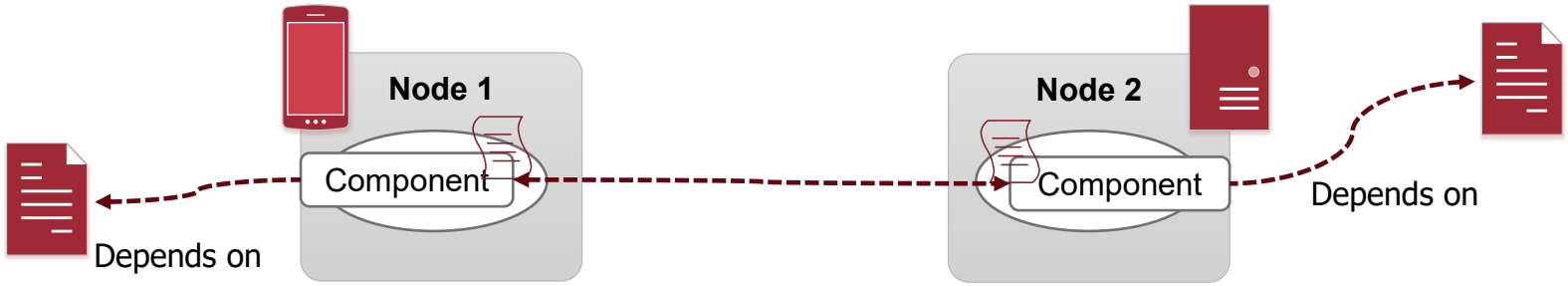


Architectural Model - Communicating entities

Problem-oriented perspective: instead of focusing on system model's point of view, focus on the developer perspective

Components in DS:

- Specified by an interface and dependencies to other components
- More complete contract for system construction

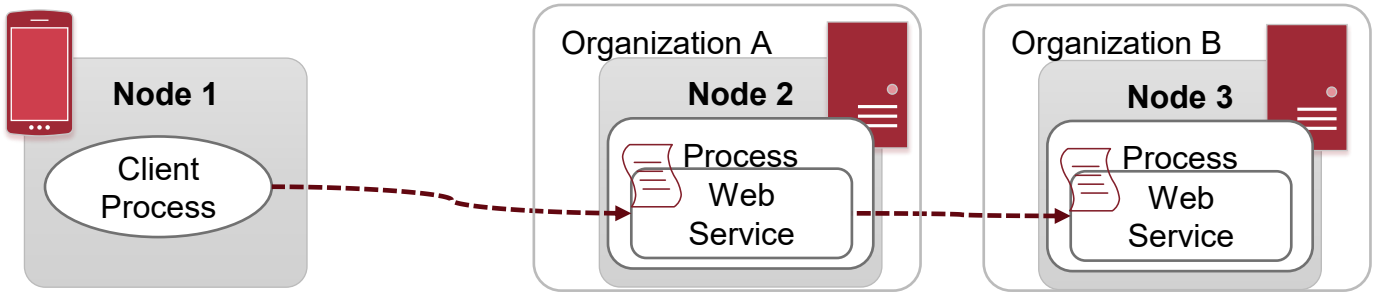


Architectural Model: Communicating entities

Problem-oriented perspective: instead of focusing on system model's point of view, focus on the developer perspective

Web Services:

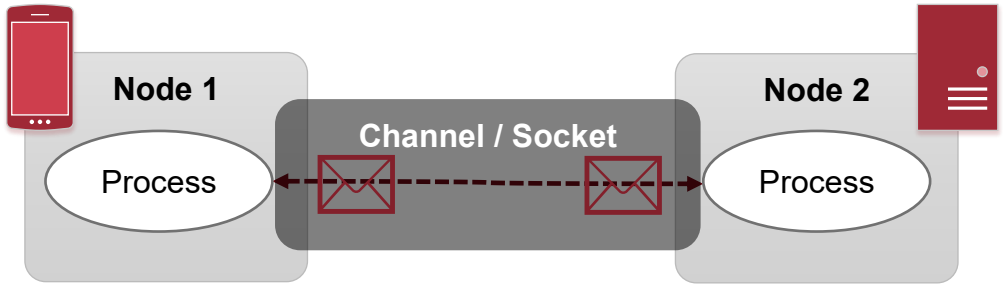
- Similar to objects and components in terms of encapsulation of behavior using interfaces
- Based on Web technologies
- Cross organizational boundaries, viewed as complete services in their own

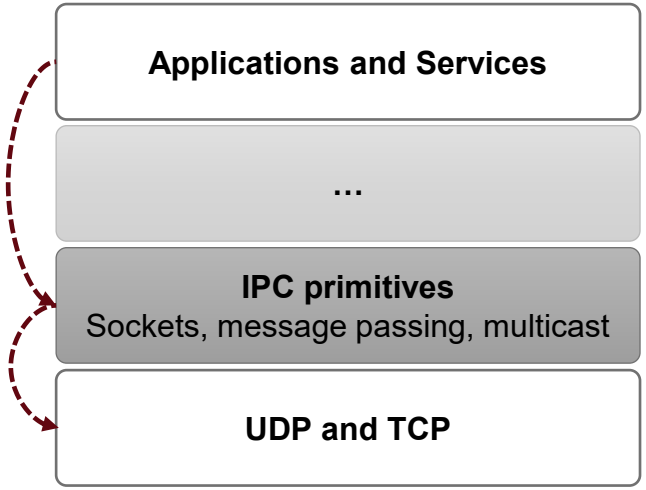


Architectural Model: Communication paradigms

Inter-process Communication (IPC)

- Low-level support for basic messaging primitives
- Offered by OS and Internet protocols (i.e. socket programming)
- Foundation for all other paradigms

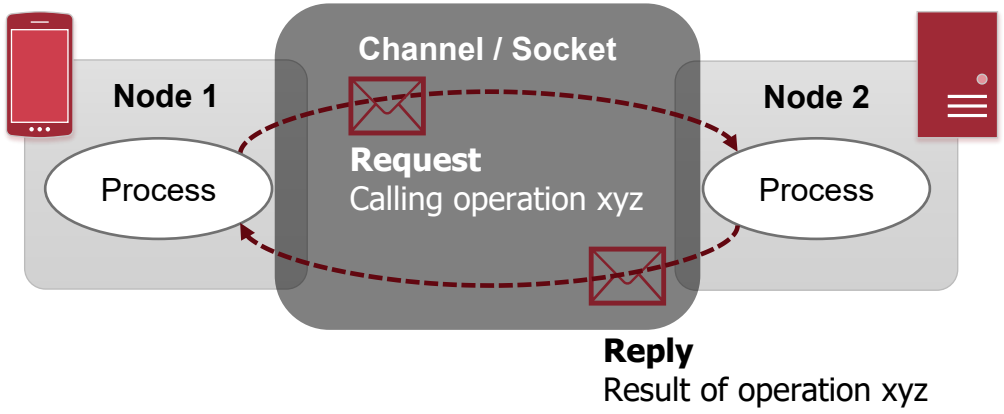


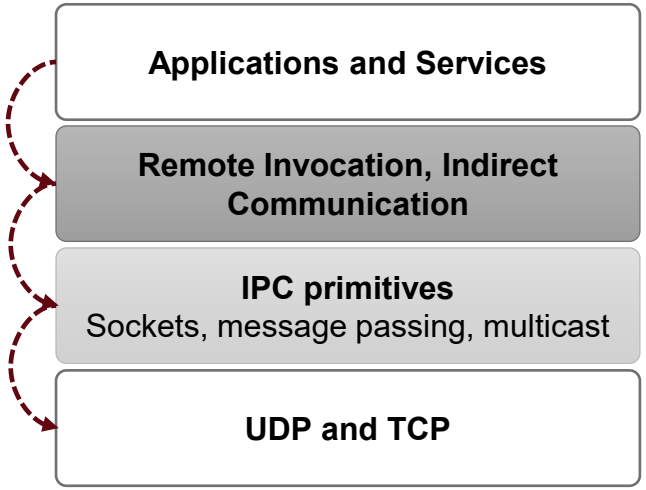


Architectural Model: Communication paradigms

Remote Invocation

- Two-way exchange between communicating entities
- Calling of a remote operation, procedure or method
- Examples:
  - Request-Reply
  - Remote Procedure Calls (RPC)
  - Remote Method Invocation (RMI)

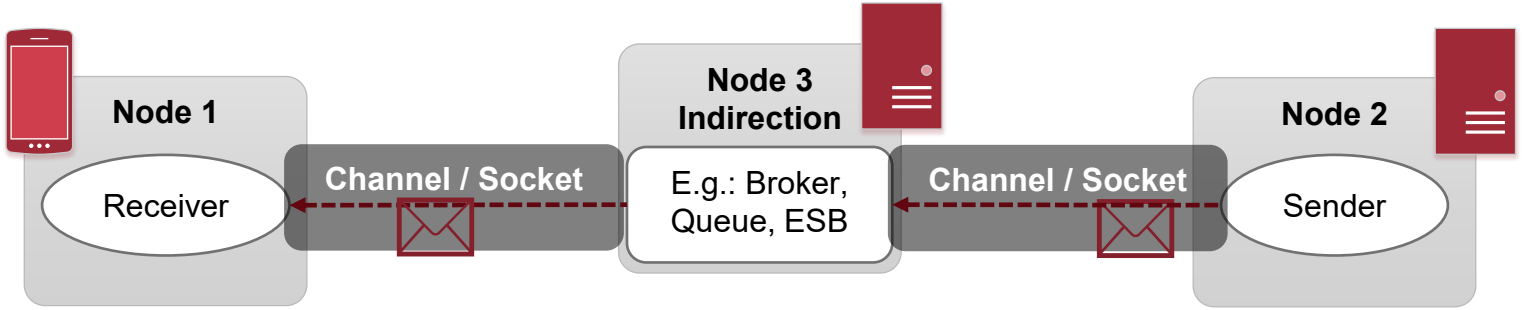




Architectural Model: Communication paradigms

Indirect Communication

- Decouples sender and receiver
- Time decoupling: sender and receiver do not need to exist at the same time
- Space decoupling: senders do not need to know who they are sending to
- Achieved by introducing an indirection between sender and receiver
- Examples: Message queues, publish / subscribe systems



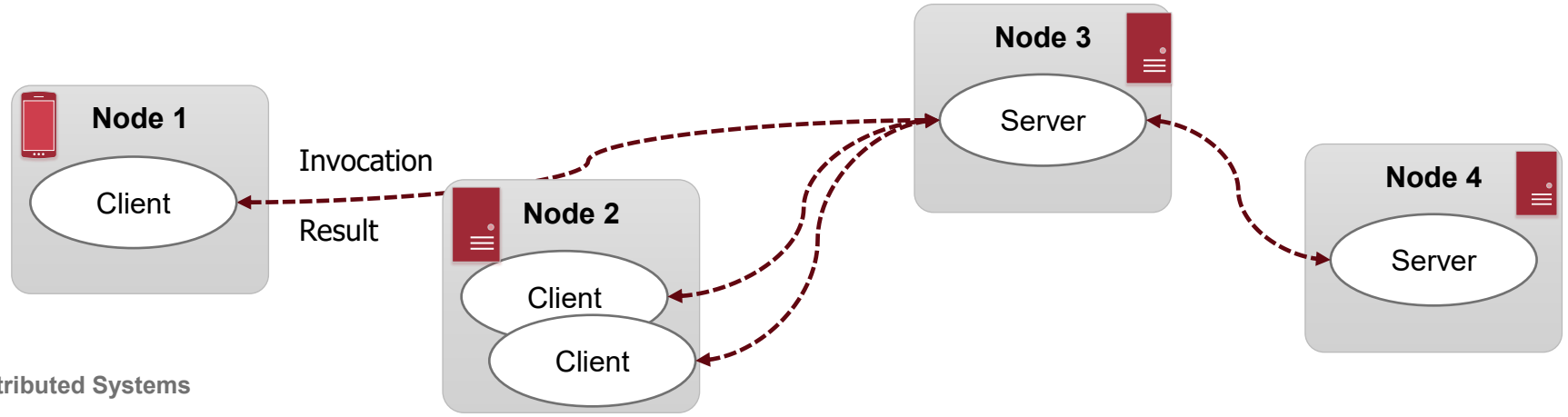
Architectural Model: Roles and Responsibilities

Entities can take the role of a client, a server or both

The role assignment regards the process, not the node!

Client-Server:

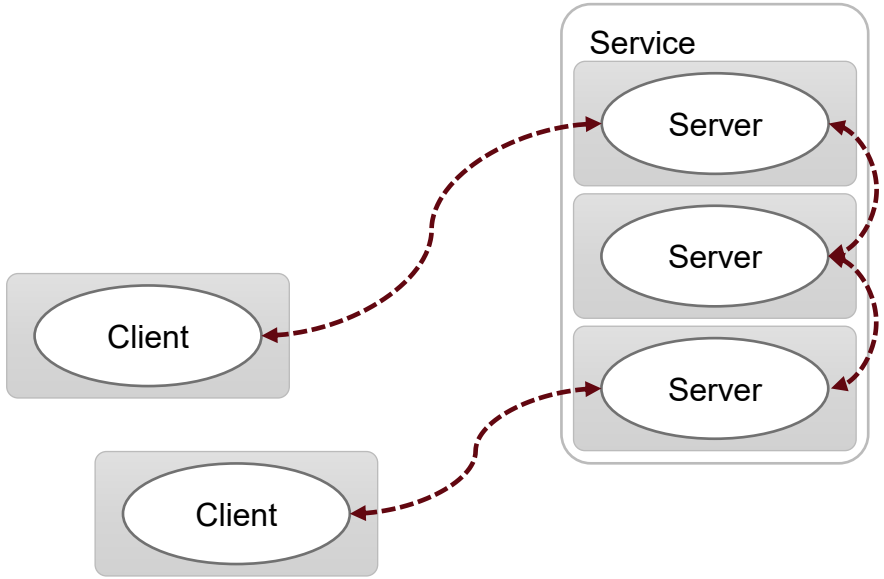
- Entities take the role of a client or server
- Servers may be clients of other servers



Architectural Model: Roles and Responsibilities

Peer-to-Peer

- No distinction between clients and servers
- All entities (peers) run the same program, offer the same interfaces and interact cooperatively
- Communication pattern between peers depends on application requirements



Architectural Model

Placement: Where to execute certain code, where to store the data → Crucial to performance, reliability and security of DS

- Considers communication patterns, reliability of machines, current load, quality of communication links, …

Placement Strategies: Mapping services → multiple servers

- Load balancing, partitioning, replication, e.g. multiple web servers manage their own set of resources
- Caching: Storing recently used data close to the client, e.g. proxy server
- Mobile Code: Additional code is downloaded and executed locally
- Agents: running program including code and data travels from node to node

Fundamental Models

Give an abstract, simplified and consistent description of an aspect when designing a distributed system:

- Interaction Model: Considers the structure and sequencing of the communication between the entities of the distributed system
- Failure Model: Considers the ways in which a system may fail to operate correctly
- Security Model: Considers how the system is protected against attempts to interfere with its correct operation or to steal its data

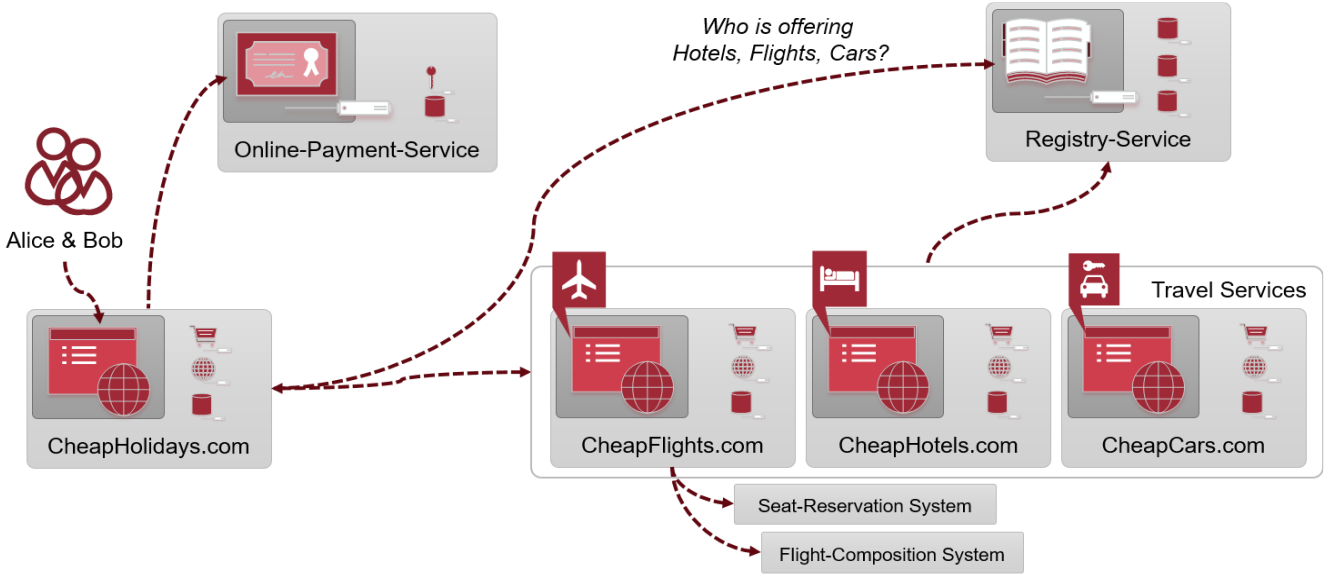
Interaction Model

How to describe the complex interaction of processes in a distributed system?

Which aspects affect the interaction of processes?

Consider our guiding thread example – What assumption can be made?

How to describe the interaction of the services?



Interaction Model

We know:

- The steps to be taken by each of the processes of which the system is composed (i.e. the operations necessary to complete Alice and Bob's booking request)
- The messages to be transmitted between the processes to transfer information and coordinate activity

We do not know:

- The rate at which each process proceeds
- The timing of the transmission of the messages
- The state is difficult to describe → each process and message can fail independently

Interaction Model

Significant factors affecting interacting processes:

- Performance of communication channels:
  - Latency: the delay between the start of a message‘s transmission from one process and the beginning of its receipt by another
  - Bandwidth: the total amount of information that can be transmitted over the channel in a given time
  - Jitter: the variation in the time taken to deliver a series of messages (e.g. important for multimedia streaming)

Interaction Model

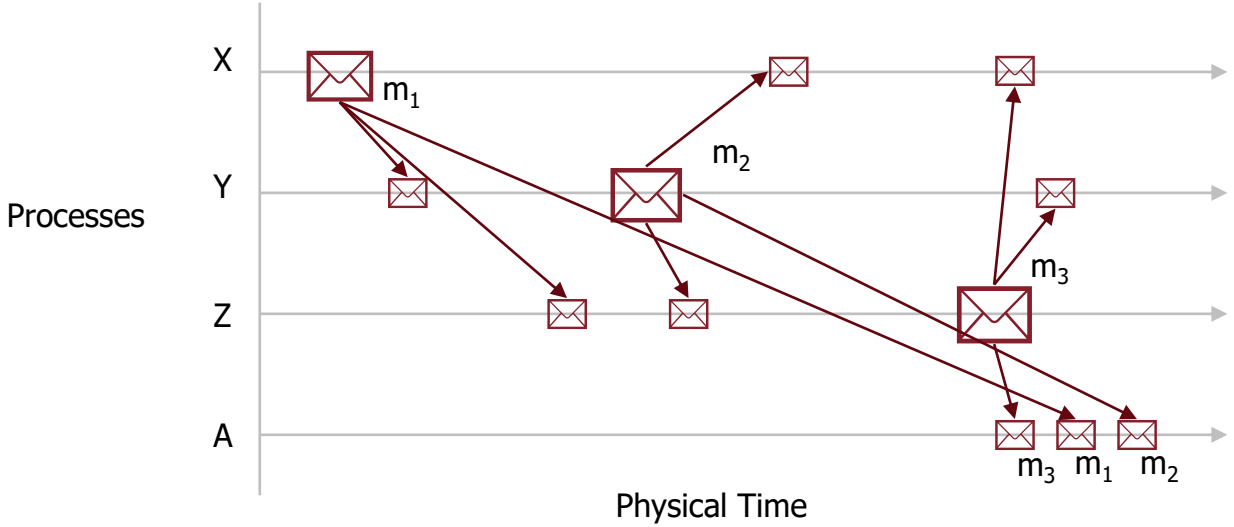
Significant factors affecting interacting processes

- Performance of communication channels
- Computer clocks and timing:
  - Processes can associate time stamps with events (i.e. when sending a message) using local clocks
  - Clocks drift from perfect time
  - Drift rates differ
  - Important with respect to event ordering!

Interaction Model – Event Ordering

Ordering required to describe whether an event at one process occurred before, after or concurrently with another event at another process

Due to delay of communication channels, other mechanisms required to preserve correct order of events: process A sees m3, m1, m2 instead of m1, m2, m3



Interaction Model – Synchronous communication

Synchronous DS defines following bounds:

- Known lower and upper bounds for each execution step of a process
- Each transmitted message is received within a known bounded time
- Each local clock has a known drift rate

Synchronous DS use timeouts to detect failures

- Upper and lower bounds, but very difficult to arrive at realistic values
- Synchronous DS often used for modeling and simulation purposes
- Although difficult to realize in the real world, synchronous DS can be built (e.g. hard real-time applications)

Interaction Model - Asynchronous

Most DS do not meet the requirements of the synchronous model (e.g. the Internet) → alternative model is required

Asynchronous DS make no assumptions regarding bounds on:

- Process execution speed
- Message transmission delays
- Clock drift rates

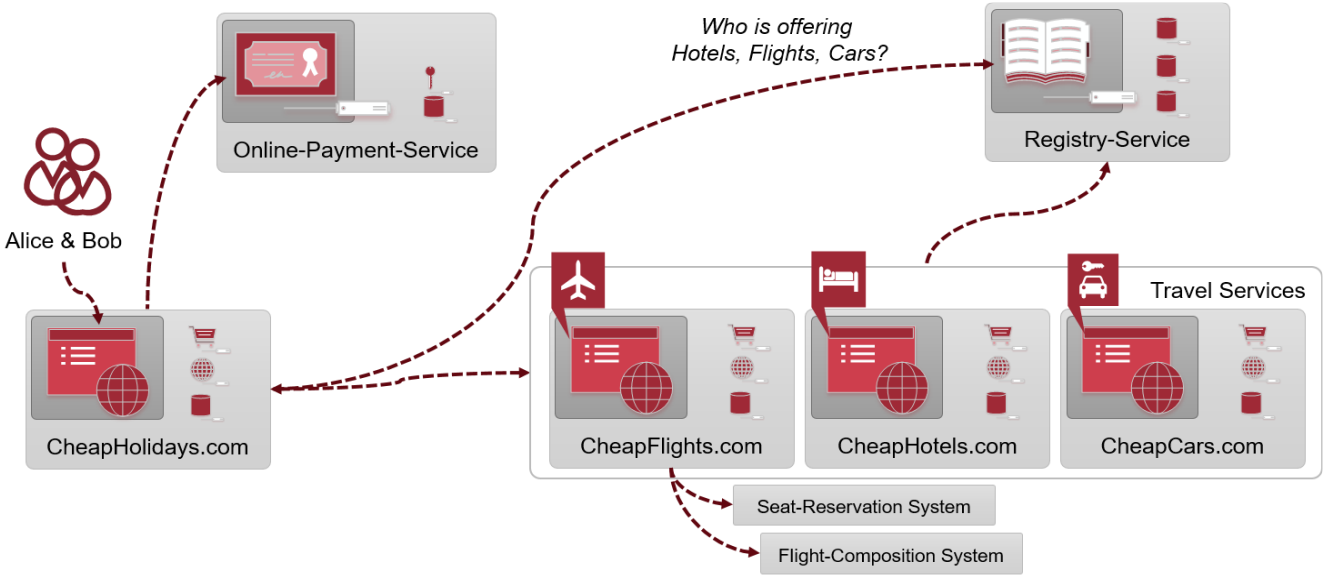
Most DS asynchronous because of shared CPUs and communication channels:

- Being aware of the asynchronous nature of a DS and the resulting implications helps us to design our algorithms appropriately

Failure Model

Defines the ways in which failures may occur

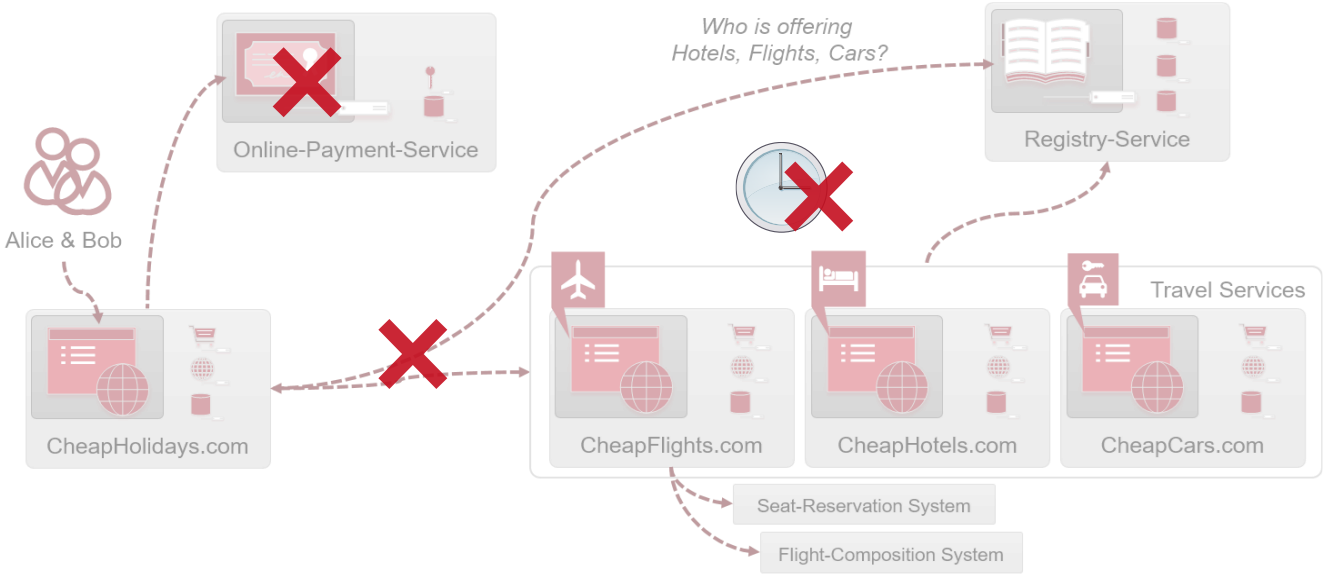
- Raise awareness of possible types of failures
- Provide an understanding of the effects of failures
- Consider our guiding thread example – What failures may occur?



Failure Model

Defines the ways in which failures may occur

- Raise awareness of possible types of failures
- Provide an understanding of the effects of failures
- Consider our guiding thread example – What failures may occur?



Failure Model

DS distinguish between failures of processes and channels

Further classification of failures:

- Omission failures: A process or a communication channel fails to perform actions it is supposed to do
- Arbitrary (Byzantine) failures:
  - Worst possible failure semantic, any type of failure can occur, difficult to detect
  - A process omits intended processing steps or takes unintended ones
  - A channel may deliver non-existent messages, real messages twice, or may corrupt the contents

Failure Model

DS distinguish between failures of processes and channels

Further classification of failures:

- Omission failures
- Arbitrary (Byzantine) failures
- Timing failures:
  - Only applicable for synchronous distributed systems
  - Defined bounds are violated (clock drift rate, interval between two processing steps, message transmission)

Failure Model – Failure Classes

Class of failure	Affects	Description
Fail-stop	Process	Process halts and remains halted. Other processes may detect this.
Crash	Process	Process halts and remains halted. Other processes may not be able to detect this state.
Omission	Channel	A message inserted in an outgoing message buffer never arrives at the other end's incoming message buffer.
Send-omission	Process	A process completes a send operation but the message is not put in its outgoing message buffer.
Receive-omission	Process	A message is put in a process's incoming message buffer, but that process does not receive it.
Arbitrary (Byzantine)	Process or Channel	Process/channel send/transmit arbitrary messages at arbitrary times or commit omissions; a process may stop or take an incorrect step.

Failure Model - Masking

Knowledge of the failure characteristics of a component can be used to design a new component that masks the failures of the component it depends on

- Example: multiple servers that hold replicas of data can continue to provide a service even if one of them crashes

Components can mask a failure by hiding it or by converting it into a more acceptable type:

- Omission failure hidden by introducing a protocol that retransmits messages
- Arbitrary message corruption failure can be converted into an omission failure by introducing checksums

Failure Model - Reliability

Although omission failures are likely to occur, communication channels can mask some of these failures

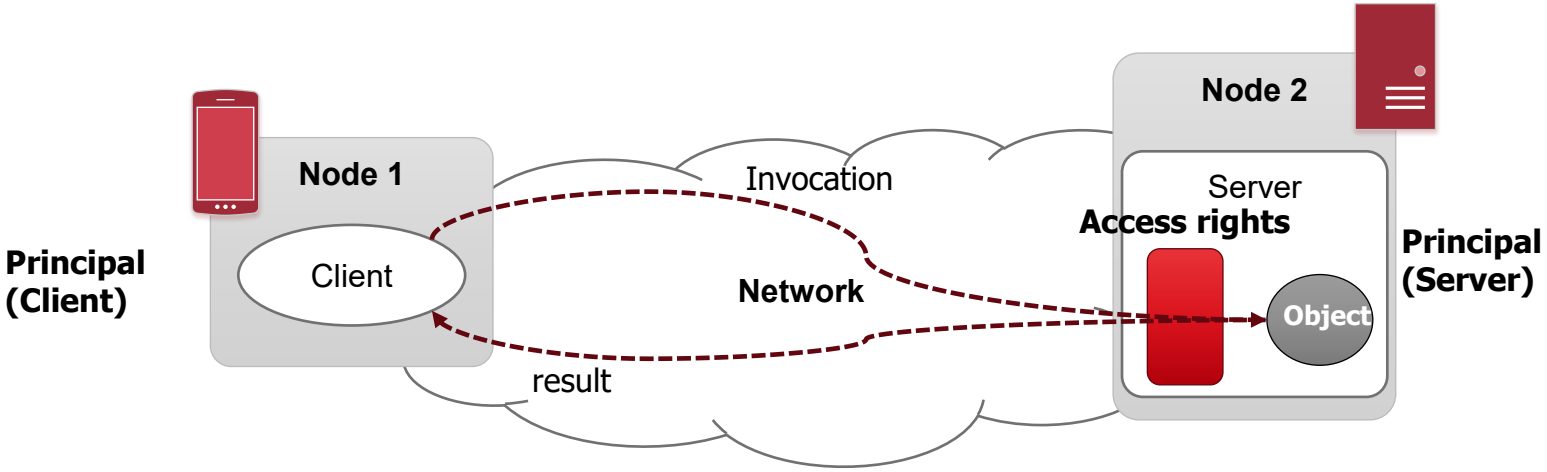
- Reliable communication is defined in terms of validity and integrity
- Validity: Any message in out-bound buffer is eventually delivered to in-bound buffer
- Integrity: The received message is identical to the one send, no message is delivered twice
- Threats to integrity: Protocols that retransmit messages but do not drop duplicates
  - Malicious users replay old messages, modify messages, or inject fake messages

Security Model

Shared resources need to be protected

Definition by Coulouris:

– „security of a distributed system can be achieved by securing the processes and the channels used for their interactions and by protecting the objects that they encapsulate against unauthorized access."



Security Model

How to protect processes and channels?

- Know your enemy → security model provides understanding for vulnerable parts of DS
- An enemy can send any message to any process, read or copy messages

Possible threats to processes:

- Designed to receive messages from any other processes → how to determine identity
- Server: is the principal allowed to perform the action?
- Client: is the source of the result authentic?

Possible threats to communication channels:

- Enemy can copy, alter or inject messages → threat to privacy and integrity
- Save message copies (replay attack, e.g. resending message for money transfer!)

Security Model

Defeating security threats

- Cryptography and shared secrets
- Authentication (based on shared secrets and encryption)
- Secure channels utilizing encryption and authentication (processes know identity of the principal on whose behalf the other one is executing, messages include time stamps to prevent reply-attacks)

System Models - Summary

Models allow for a better understanding of DS and help to design more reliable, fault tolerant, and efficient systems

- e.g.: use replicas and multiple servers to operate critical services more reliable
- Most problems can be solved by appropriate design, yet there are limitations!

Questions ?